Stoquify Agent and Skill Leverage Audit

Date: 2026-07-21 Scope: Stoquify local codebase inspection plus current public GitHub and primary-source research into agents, agent frameworks, skill systems, and workflow tools that can raise Stoquify's professional level with the least code and highest leverage.

Executive Decision

Stoquify should not adopt a large agent platform as a replacement architecture. The codebase already has strong control-plane assets: Next.js/React/Prisma application structure, module surface inventory, RBAC gates, close-assurance reports, release gates, graphify knowledge graphs, and many existing Stoquify-specific Codex skills.

The highest-leverage move is to turn those assets into a disciplined, report-first agent operating layer:

1. Standardize Stoquify's local skills using the emerging `SKILL.md` pattern.
2. Add a read-only Stoquify MCP server that exposes module inventory, graphify outputs, package scripts, release gates, and `what-next` evidence.
3. Add agent evaluation and prompt regression gates before any agent affects real finance, payroll, inventory, evidence, or close data.
4. Use TypeScript-first in-app agent tooling only where it fits the existing Next.js surface.
5. Keep statutory, payroll, finance, billing, module entitlement, and close-assurance agents report-only until their authority boundaries and evidence gates are explicitly certified.

Short version: package Stoquify's existing operating knowledge first; only then attach smarter agents to it.

Local Stoquify Findings

The local inspection shows a mature but complex operational application:

- Stack: Next.js 15, React 19, Prisma 6, server actions, services, Jest, Playwright, policy gates, and release-readiness scripts.
- Domains present: finance, accounting, payroll, HRIS, inventory, POS, purchasing, payments, compliance, assurance, evidence, modules, roles, analytics, manager action center, and owner war room.
- Module inventory: 20 catalog modules, 367 discovered surfaces, 339 enforcement candidates, 334 mapped candidates, 4 missing permissions, 5 unmapped, and 10 active gaps in the latest module surface inventory.
- Graph assets: graphify outputs already map thousands of nodes and relationships across app routes, actions, services, hooks, components, and types.
- Evidence posture: close assurance and payroll proof reports are development-ready in places, but production authority integrations and qualified review remain blockers in statutory and country-pack flows.
- Current advantage: Stoquify already behaves like an operating system for trusted business execution. External agents should amplify that system, not bypass it.

Evaluation Rubric

Each candidate was scored qualitatively against Stoquify's needs:

- Leverage: can it improve a large surface with limited code?
- Fit: does it match Next.js, TypeScript, Prisma, RBAC, evidence, and report-driven workflows?
- Safety: can it run read-only, report-only, or behind gates?
- Professional lift: does it improve reliability, auditability, developer velocity, or client trust?

- Complication risk: does it introduce runtime burden, vendor lock-in, hosting complexity, or unsafe automation pressure?

Worth categories:

- Use now: high leverage, low blast radius, clear fit.
- Pilot: promising, but should start narrow and report-only.
- Later: useful after foundations mature.
- Avoid for now: too much complexity, overlap, or architecture churn for current Stoquify needs.

Top Recommendations

1. Use a Stoquify skill suite first.

Package recurring work into strict local skills: RBAC enforcement pass, module surface inventory refresh, release evidence review, payroll proof readout, finance close-readiness review, and security gate remediation.

2. Add a read-only Stoquify MCP server.

Expose `what-next`, `graphify-out`, package scripts, module inventory JSON, and selected release-gate outputs to agents. This gives agents context without giving them write access to production-sensitive flows.

3. Add evaluation and observability before in-product AI.

Use Promptfoo, DeepEval, and Langfuse-style workflows to test agent answers, tool calls, prompt drift, red-team cases, and regression behavior.

4. Keep the first in-app assistant narrow.

A good first assistant is not "do everything." It should be a read-only evidence analyst that explains finance/payroll/inventory/close blockers and recommends the next gate-safe action.

5. Treat browser and coding agents as developer accelerators, not production operators.

Aider, OpenHands, Browser-use, Stagehand, and Playwright MCP can help test, inspect, and repair flows, but they should not own finance, payroll, statutory, or entitlement writes.

20 Agent and Framework Candidates

#	Agent / Framework	Best Stoquify Use	What It Brings	Pros	Cons / Complications	Worth
1	OpenAI Agents SDK	Report-only operational analysts and controlled multi-step workflows	Agents, tools, handoffs, guardrails, sessions, tracing, realtime, and human-in-the-loop concepts	Clean conceptual model, strong guardrail fit, good for finance/payroll evidence agents	Python-first if the app layer stays TypeScript; production actions still need local policy gates	Pilot
2	Microsoft Agent Framework	Enterprise-style durable multi-agent workflows	Python/.NET framework with multi-agent workflows, A2A, MCP, and durable hosting patterns	Strong enterprise framing; good for future regulated workflows	Heavier than Stoquify needs today; learning curve	Later
3	Google Agent Development Kit	Multi-agent research and evidence workflows	Code-first Python agents, tools, MCP/OpenAPI, HITL, eval, deployment options, A2A	Good for experimental agent orchestration and cross-tool research	Another Python runtime; not naturally Next-native	Pilot
4	LangGraph	Stateful, long-running agent workflows	Graph-based execution, memory/state, human review, deterministic flow design	Strong fit for approval chains, close workflows, and gated review loops	Adds a framework layer; needs discipline to avoid over-modeling	Pilot
5	Mastra	TypeScript agent runtime for Stoquify's Next.js world	Agents, workflows, MCP, evals, observability, model routing	Best fit among broader agent frameworks for a TypeScript product team	Platform surface is still another abstraction; start small	Pilot
6	Vercel AI SDK	In-app AI UI and server-side AI endpoints	Streaming, provider abstraction, tools, React/Next patterns	Very strong fit for existing Next.js; least friction for product UX	It is not a full governance system by itself	Use now for narrow UI
7	PydanticAI	Typed Python agents for report production	Pydantic validation, dependency injection, model-agnostic agents, observability options	Excellent for structured, audited report outputs	Python service boundary required; less natural for frontend UX	Pilot

#	Agent / Framework	Best Stoquify Use	What It Brings	Pros	Cons / Complications	Worth
8	LlamaIndex	Evidence and document retrieval agents	Document ingestion, RAG, indexes, many integrations, LlamaParse ecosystem	Useful for receipts, statutory docs, evidence packs, and knowledge retrieval	RAG complexity and data governance can grow quickly	Pilot
9	CrewAI	Multi-role agent crews for analysis workflows	Lightweight Python multi-agent orchestration	Easy to conceptualize specialist agents: finance reviewer, payroll reviewer, release reviewer	Role-playing agents can become theatrical unless strongly grounded in tools	Later
10	CopilotKit	Embedded product copilot UX	React/Angular agent UI, generative UI, shared state, human-in-the-loop	Strong if Stoquify wants a polished in-app copilot	Adds frontend framework conventions; must respect RBAC and evidence gates	Pilot
11	OpenHands	Developer coding agent	Autonomous repo work through CLI/GUI/cloud options	Useful for developer velocity and issue remediation	Too broad for production operations; must be sandboxed and reviewed	Pilot for dev only
12	Aider	Git-aware pair programming	Terminal coding agent with repo map, git integration, test/lint loops	Very high leverage for focused code changes with low ceremony	Still needs human review; can touch many files if prompts are loose	Use now for dev only
13	SWE-agent / mini-SWE-agent	Issue-fixing research agent	Software engineering agent with shell/computer interface patterns	Good benchmark and repair-agent reference	Original project points users toward mini-SWE-agent; not ideal as product runtime	Later
14	Browser-use	Browser automation agents	Python browser automation with custom tools and cloud options	Useful for end-to-end smoke, onboarding checks, and exploratory testing	Browser agents are brittle on high-value workflows without strict assertions	Pilot
15	Stagehand	Controlled browser automation	Natural-language plus code browser actions, extraction, self-healing patterns	More deterministic than pure browser agents; good for smoke tests and UX audits	Browserbase/cloud assumptions may add vendor and infra concerns	Pilot
16	Playwright MCP	Agent-accessible browser testing	MCP browser server using accessibility snapshots	Strong fit for existing Playwright discipline; can let agents inspect UI without screenshots only	Should complement, not replace, deterministic Playwright tests	Use now
17	Dify	Visual agent workflow platform	App builder, workflows, observability, plugins, knowledge tooling	Useful for quick internal prototypes and non-developer workflow demos	A parallel platform can drift from Stoquify code governance	Later
18	n8n	Workflow automation and AI agents	Self-hostable automation, many integrations, human-in-the-loop, observability	Excellent for back-office automations around evidence collection, notifications, and approvals	License and operational ownership need review; avoid core ledger logic	Pilot around integrations
19	Temporal AI / durable agents	Long-running reliable operations	Durable execution, retries, workflows, agent integration patterns	Best fit for eventually reliable payroll/close/provider jobs	Infrastructure cost; should not be introduced for small workflows	Later, high value
20	Microsoft AutoGen	Multi-agent reference architecture	Mature research lineage for agent conversations and tool use	Useful for learning patterns and comparing concepts	Official direction has shifted toward Microsoft Agent Framework; not a new Stoquify foundation	Avoid for new build

20 Skill and Workflow Candidates

#	Skill / Workflow	Best Stoquify Use	What It Brings	Pros	Cons / Complications	Worth
1	Anthropic-style Agent Skills	Standardize Stoquify runbooks as reusable `SKILL.md` folders	Portable instructions, scripts, and resources for agents	Directly matches Stoquify's existing skill-heavy operating model	Needs curation and versioning; bad skills can codify bad habits	Use now
2	skills.sh / Vercel Labs skills	Installable and shareable skill packaging	CLI-style skill distribution for multiple coding agents	Good way to package Stoquify internal skills cleanly	External catalog quality varies; vet before adopting	Use now for packaging pattern
3	Microsoft Skills catalog	Enterprise and Azure-oriented agent instructions	Skills, MCP, custom agents, and AGENTS.md patterns	Useful reference for professional skill structure	Many skills are Microsoft/Azure-specific; not all relevant	Use as reference
4	Temporal Developer Skill	Better durable workflow code	Agent guidance for Temporal application development	Useful if Stoquify adopts durable payroll/close workflows later	Premature if Temporal is not selected	Later
5	Vercel AI SDK skill	Faster Next.js AI implementation	Agent instructions for Vercel AI SDK usage	Strong fit for Stoquify's frontend stack	Only valuable if Stoquify builds in-app AI surfaces	Use now when building AI UI
6	MCP server/client skill	Read-only context layer for Stoquify agents	Standard way to expose tools, resources, and prompts to agents	Highest-leverage bridge between codebase evidence and agent reasoning	Tool permission boundaries must be strict	Use now
7	Promptfoo evaluation skill	LLM regression and red-team gates	CLI/library for prompt tests, model comparisons, and attacks	Strong fit for release gates and AI assurance	Requires scenario writing and maintained fixtures	Use now
8	DeepEval agent evaluation skill	Test agent outputs, tools, RAG, and safety	Pytest-like LLM metrics and CI/CD flow	Good for structured evidence-agent tests	Python dependency and metric selection work	Pilot

#	Skill / Workflow	Best Stoquify Use	What It Brings	Pros	Cons / Complications	Worth
9	Langfuse observability skill	Trace prompts, outputs, datasets, and evals	Open-source LLM observability and prompt management	Good for auditability and prompt drift control	Must avoid logging sensitive payroll/finance data	Pilot
10	Guardrails AI validation skill	Validate agent inputs and outputs	Validators, structured data, and guardrails server	Strong fit for controlled report outputs	Extra runtime layer; overlapping choices with Instructor/BAML	Pilot
11	Instructor structured-output skill	Pydantic-validated LLM responses	Typed extraction, retries, validation, streaming	Excellent for evidence summaries and deterministic JSON outputs	Python-centric; not a complete governance layer	Use now for Python reports
12	BAML prompt-contract skill	Type-safe prompt and output contracts	Prompt-as-code, generated clients, model-agnostic structured outputs	Strong professional lift for AI contracts	Adds another DSL and build step	Pilot
13	Outlines constrained-output skill	Guaranteed JSON/schema/regex-style outputs	Constrained generation for structured results	Useful for high-certainty report fields	Model/runtime compatibility needs review	Later
14	Docling evidence-ingestion skill	Parse documents into evidence pipelines	PDF/DOCX/PPTX/XLSX/HTML/document parsing with table/layout support	Strong fit for invoices, receipts, payslips, statutory docs, XBRL-style material	Evidence classification and privacy controls still need Stoquify logic	Pilot
15	Graphiti temporal-memory skill	Build temporal knowledge graphs for decisions and evidence	Provenance-aware, time-aware graph memory for agents	Excellent fit for "what changed, who approved, what evidence supports it"	Requires graph database and data governance	Pilot
16	Soda Core data-contract skill	Data quality checks for finance/payroll/inventory tables	YAML data contracts and checks across SQL systems	Good for ledger, stock, payroll, and reporting trust	Another check layer to maintain	Use now
17	Great Expectations quality skill	Data validation and expectation suites	Broad data quality framework	Useful for analytics and reporting confidence	Can become heavy; choose focused checks only	Pilot
18	OpenLineage / Marquez lineage skill	Track report and data provenance	Open lineage standard and reference server	Strong for assurance, report trust, and evidence traceability	Requires instrumentation and process ownership	Later
19	Semgrep + CodeQL security skill	Static analysis for app/security rules	Custom code rules, CodeQL queries, CI findings	Strong fit for RBAC, server-action boundaries, unsafe Prisma patterns	Tuning required to reduce noise	Use now
20	OSV-Scanner + Renovate + zizmor + Trivy release skill	Supply-chain and CI hardening	Vulnerability/license scanning, dependency PRs, GitHub Actions analysis, container/IaC/security scanning	High leverage with little product code; improves release confidence	Requires version pinning, policy tuning, and careful Trivy release hygiene	Use now

Decision Buckets

Use now:

- Stoquify local skill standardization.
- Read-only MCP over `what-next`, `graphify-out`, package scripts, and module inventory.
- Vercel AI SDK for narrow Next.js AI UI.
- Playwright MCP to enhance browser inspection.
- Promptfoo for prompt and agent regression tests.
- Soda Core for focused data contracts.
- Semgrep, CodeQL, OSV-Scanner, Renovate, zizmor, and carefully pinned Trivy.
- Aider for developer-side focused implementation, not production automation.

Pilot:

- Mastra for TypeScript-native agent workflows.
- OpenAI Agents SDK for report-only analysts and controlled handoffs.
- LangGraph for long-running stateful review flows.

- PydanticAI or Instructor for typed report production.
- Docling for evidence document ingestion.
- Graphiti for temporal decision and evidence memory.
- Browser-use or Stagehand for browser-assisted smoke and UX audits.
- Langfuse and DeepEval for observability and evaluation.
- n8n for non-core back-office integrations.

Later:

- Microsoft Agent Framework.
- Google ADK.
- Temporal durable agents.
- LlamaIndex as a broader RAG layer after evidence privacy patterns are firm.
- OpenLineage / Marquez for deeper lineage.
- Great Expectations if Soda Core is insufficient.

Avoid for now:

- Microsoft AutoGen as a new foundation, because Microsoft now positions Agent Framework as the newer path.
- Any agent with write access to finance, payroll, statutory filings, close certification, module entitlement, or billing before Stoquify has explicit tool permissions, report-only dry runs, regression tests, and human approval checkpoints.

Recommended 30-Day Pilot

Week 1: Skill foundation

- Create a first-class Stoquify skill catalog for recurring work:

- module surface inventory review - narrow RBAC enforcement - payroll proof-chain review - finance close-readiness review - release gate explanation - browser smoke triage

- Normalize each skill to a small `SKILL.md`, optional scripts, and a strict "do not cross" boundary section.

Week 2: Read-only MCP

- Build a local Stoquify MCP server with read-only resources:

- `what-next/module-surface-inventory.json` - `what-next/module-surface-inventory.md` - selected `what-next` readiness reports - `graphify-out` reports - package script index - route/action/service summaries

- Do not expose write tools in the first version.

Week 3: Evaluation gate

- Add Promptfoo cases for:

- RBAC advice must not recommend bypassing permission checks. - Payroll advice must distinguish development-ready from production-authority-ready. - Close-assurance advice must not claim legal/statutory certification unless evidence says so. - Module entitlement recommendations must remain report-only.

- Add DeepEval or Langfuse only after the Promptfoo fixture set is stable.

Week 4: First product-facing assistant

- Build a narrow "Evidence Readiness Analyst" in the existing Next.js app.
- Capabilities:

- read evidence state - explain blockers - cite source report - recommend next safe action - produce exportable summary

- Prohibited:

- no ledger writes - no payroll writes - no statutory submission - no entitlement activation - no provider payment action

Stoquify-Specific Agent Concepts Worth Building

1. Module Surface Steward

Reads module inventory, reports unmapped and missing-permission gaps, proposes narrow next enforcement passes, and never changes entitlement state.

2. RBAC Patch Planner

Reads action/service/route mappings, proposes the smallest enforcement patch and test set, then waits for a developer to implement or approve.

3. Payroll Proof Reviewer

Explains payroll payment/declaration proof readiness, separating synthetic readiness, sandbox readiness, and real authority readiness.

4. Finance Close Evidence Analyst

Reviews close blockers, ledger tie-outs, evidence packs, and report trust state without certifying legal compliance.

5. Stock-to-Cash Drift Analyst

Compares inventory, POS, cash drawer, purchasing, and finance surfaces for workflow leakage or missing evidence.

6. Release Gate Narrator

Turns CI, readiness, and policy-gate outputs into executive-friendly release notes with exact blockers and evidence references.

7. Browser Smoke Assistant

Uses Playwright MCP or Stagehand to inspect high-value pages and produce issue reports with screenshots, routes, and reproduction steps.

8. Data Contract Watcher

Runs focused Soda Core checks for finance, payroll, stock, and evidence tables before reports are trusted.

Implementation Guardrails

- Report-only first. Every new agent should initially produce recommendations, not mutate business state.
- Read-only MCP first. Context access is valuable; write tools are where risk begins.
- Strict permission vocabulary. Reuse Stoquify's module and RBAC language; do not invent parallel access concepts.
- Human approval for high-value operations. Payroll, statutory, finance close, entitlement, billing, and provider actions require explicit approval paths.

- Evidence citations are mandatory. Any agent answer about readiness, blockers, certification, or release state must cite the local source report or generated evidence.
- No statutory overclaiming. Development readiness, sandbox readiness, qualified review, and production authority readiness must remain separate.
- No hidden autonomy. Agents should show the input evidence, decision rule, and proposed next action.
- Smallest viable integration. Prefer skills, MCP resources, and CI checks before adopting platform infrastructure.

Sources Reviewed

Primary local sources:

- `package.json`
- `README.md`
- `what-next/module-surface-inventory.md`
- `what-next/ci-release-readiness.md`
- `what-next/AQSTOQFLOW_CLOSE_ASSURANCE_CENTER_IMPLEMENTATION_REPORT_2026-07-20.md`
- `what-next/payroll/AQSTOQFLOW_HRIS_PAYROLL_PAYMENTS_DECLARATIONS_PROOF_2026-07-20.md`
- `what-next/ui-ux/stoquify-landing-release-gate-2026-07-19.md`
- `graphify-out/ORDERED_GRAPHIFY_RUN_2026-07-14.md`
- `graphify-out/GRAPH_REPORT.md`

Primary internet and GitHub sources:

- OpenAI Agents SDK: https://github.com/openai/openai-agents-python and https://openai.github.io/openai-agents-python/
- Microsoft Agent Framework: https://github.com/microsoft/agent-framework and https://learn.microsoft.com/en-us/agent-framework/
- Google ADK: https://github.com/google/adk-python and https://google.github.io/adk-docs/
- LangGraph: https://github.com/langchain-ai/langgraph and https://docs.langchain.com/oss/python/langgraph/overview
- Mastra: https://github.com/mastra-ai/mastra and https://mastra.ai/
- Vercel AI SDK: https://github.com/vercel/ai and https://ai-sdk.dev/docs/
- PydanticAI: https://github.com/pydantic/pydantic-ai and https://ai.pydantic.dev/
- LlamaIndex: https://github.com/run-llama/llama_index and https://docs.llamaindex.ai/
- CrewAI: https://github.com/crewAIInc/crewAI
- CopilotKit: https://github.com/CopilotKit/CopilotKit and https://docs.copilotkit.ai/
- OpenHands: https://github.com/All-Hands-AI/OpenHands and https://docs.all-hands.dev/
- Aider: https://github.com/Aider-AI/aider and https://aider.chat/
- SWE-agent: https://github.com/SWE-agent/SWE-agent and https://swe-agent.com/latest/
- Browser-use: https://github.com/browser-use/browser-use and https://docs.browser-use.com/

- Stagehand: https://github.com/browserbase/stagehand and https://docs.stagehand.dev/
- Playwright MCP: https://github.com/microsoft/playwright-mcp
- Dify: https://github.com/langgenius/dify and https://docs.dify.ai/
- n8n: https://github.com/n8n-io/n8n and https://docs.n8n.io/
- Temporal AI documentation: https://docs.temporal.io/ai
- Microsoft AutoGen: https://github.com/microsoft/autogen
- Anthropic Agent Skills: https://github.com/anthropics/skills
- skills.sh / Vercel Labs skills: https://skills.sh/ and https://github.com/vercel-labs/skills
- Microsoft Skills: https://github.com/microsoft/skills
- Temporal Developer Skill: https://github.com/temporalio/skill-temporal-developer
- MCP TypeScript SDK: https://github.com/modelcontextprotocol/typescript-sdk
- MCP Python SDK: https://github.com/modelcontextprotocol/python-sdk
- Promptfoo: https://github.com/promptfoo/promptfoo and https://www.promptfoo.dev/
- DeepEval: https://github.com/confident-ai/deepeval and https://deepeval.com/docs/
- Langfuse: https://github.com/langfuse/langfuse and https://langfuse.com/docs
- Guardrails AI: https://github.com/guardrails-ai/guardrails
- Instructor: https://github.com/567-labs/instructor and https://python.useinstructor.com/
- BAML: https://github.com/BoundaryML/baml and https://docs.boundaryml.com/
- Outlines: https://github.com/dottxt-ai/outlines and https://dottxt-ai.github.io/outlines/latest/
- Docling: https://github.com/docling-project/docling and https://docling-project.github.io/docling/
- Graphiti: https://github.com/getzep/graphiti
- Soda Core: https://github.com/sodadata/soda-core and https://docs.soda.io/
- Great Expectations: https://github.com/great-expectations/great_expectations and https://docs.greatexpectations.io/
- OpenLineage: https://github.com/OpenLineage/OpenLineage and https://openlineage.io/docs/
- Marquez: https://github.com/MarquezProject/marquez
- Semgrep: https://github.com/semgrep/semgrep and https://semgrep.dev/docs/
- CodeQL: https://github.com/github/codeql and https://codeql.github.com/docs/
- OSV-Scanner: https://github.com/google/osv-scanner
- Renovate: https://github.com/renovatebot/renovate and https://docs.renovatebot.com/
- zizmor: https://github.com/zizmorcore/zizmor and https://docs.zizmor.sh/
- Trivy: https://github.com/aquasecurity/trivy

Final Recommendation

The best professional lift for Stoquify is not "more agents everywhere." It is a controlled agent operating layer over evidence that Stoquify already produces.

Build this in order:

1. Local Stoquify skill suite.
2. Read-only Stoquify MCP.
3. Prompt and agent evaluation gates.
4. Browser-assisted smoke and UX audit tooling.
5. One narrow in-app Evidence Readiness Analyst.
6. Durable workflow agents only after the read-only layer proves useful and trusted.

This path gives Stoquify a more intelligent operating model without weakening the finance, payroll, inventory, evidence, module entitlement, and release-assurance controls that make the system valuable.